

Code Explanation

Address Change Processor Test Explanation

This test suite is designed to validate the functionality of the `AddressChangeProcessor` class. It covers various aspects of the processor, including reading staging data, applying lookups, transforming data, preparing YUYU extract, and preparing staging updates.

Overview of the Code

The test suite utilizes Pytest fixtures to create a Spark session, an instance of `AddressChangeProcessor`, and sample data for testing. It then defines several test functions to validate the different stages of the address change processing pipeline.

Step 1: Creating a Spark Session and Test Data

This step involves creating a Spark session and generating sample data for testing. The `spark` fixture creates a Spark session with a local master node, while the `sample\_data` fixture generates a DataFrame with sample data and creates temporary views for lookup tables.

```
@pytest.fixture(scope="module")
def spark():
    return SparkSession.builder
        .appName("AddressChangeProcessorTest")
        .master("local[*]")
        .getOrCreate()

@pytest.fixture(scope="module")
def sample_data(spark):
    # Create sample data and lookup tables
    schema = StructType([...])
    data = [...]
    df = spark.createDataFrame(data, schema)
    # ...
    return {
        "source_df": df,
        "yuyu_clnt_df": yuyu_clnt_df,
        "yuyuk_cln_df": yuyuk_cln_df
    }
```

Step 2: Testing the `read\_staging\_data` Method

This step tests the `read\_staging\_data` method of the `AddressChangeProcessor` class. It verifies that the method correctly reads data from the specified temporary view.

```
def test_read_staging_data(spark, processor, sample_data):
    df = processor.read_staging_data("STG_E2E_AC_TXDBH_DATA")
    assert df.count() == 3
    assert df.filter(df.T_TX_REQUEST_REQUEST_ID == "REQ001").count()
    == 1
```

Step 3: Testing the `apply\_lookups` Method

This step tests the `apply\_lookups` method, which enriches the input DataFrame with data from lookup tables. It checks if the lookup fields are present in the resulting DataFrame and verifies the lookup

values.

```
def test_apply_lookups(spark, processor, sample_data):
    source_df = sample_data["source_df"]
    enriched_df = processor.apply_lookups(source_df)
    assert "POWN_LNM" in enriched_df.columns
    assert "POWN_FNM" in enriched_df.columns
    assert "POWN_KNM" in enriched_df.columns
    # ...
```

Step 4: Testing the `transform\_data` Method

This step tests the `transform\_data` method, which applies various transformations to the enriched DataFrame. It checks if the transformation fields are present and verifies the transformed values.

```
def test_transform_data(spark, processor, sample_data):
    source_df = sample_data["source_df"]
    enriched_df = processor.apply_lookups(source_df)
    transformed_df = processor.transform_data(enriched_df)
    assert "O_NEW_ZIP" in transformed_df.columns
    assert "O_NEW_PHONE_NUMBER" in transformed_df.columns
    assert "O_KANA_NAME" in transformed_df.columns
    # ...
```

Step 5: Testing the `prepare\_yuyu\_extract` Method

This step tests the `prepare\_yuyu\_extract` method, which prepares the YUYU extract DataFrame. It checks if the expected columns are present in the resulting DataFrame and verifies the row count.

```
def test_prepare_yuyu_extract(spark, processor, sample_data):
    source_df = sample_data["source_df"]
    enriched_df = processor.apply_lookups(source_df)
    transformed_df = processor.transform_data(enriched_df)
    yuyu_df = processor.prepare_yuyu_extract(transformed_df)
    expected_columns = [...]
    for col in expected_columns:
        assert col in yuyu_df.columns
    assert yuyu_df.count() == 3
```

Step 6: Testing the `prepare\_staging\_update` Method

This step tests the `prepare\_staging\_update` method, which prepares the staging update DataFrame. It checks if the expected columns are present in the resulting DataFrame and verifies the row count and process user.

```
def test_prepare_staging_update(spark, processor, sample_data):
    source_df = sample_data["source_df"]
    enriched_df = processor.apply_lookups(source_df)
    transformed_df = processor.transform_data(enriched_df)
    update_df = processor.prepare_staging_update(transformed_df)
    expected_columns = [...]
    for col in expected_columns:
        assert col in update_df.columns
```

```
assert update_df.count() == 3
first_row = update_df.first()
assert first_row["STG_PROCESS_USERID"] == "TEST_USER"
```